UNIVERSAL CODE SCANNER

Documentación de sistema · Universal Code Scanner

Documentación de sistema

Versión analizada: **1.1.0+2**
Commit: **6d5e0b7**
Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. El sistema en una frase
2. Para quién es esta documentación
3. Tabla de contenidos
4. Datos del análisis
5. Convenciones
6. Qué se verificó ejecutando, y qué no
7. Pendiente de validar
8. Relación con la documentación previa

Portada e índice de la documentación técnica, funcional, arquitectónica y operativa del repositorio. Se escribió recorriendo el código fuente completo, no resumiendo la documentación existente: cada afirmación apunta a un archivo, una función, una prueba o un comando concreto de este repositorio.

El sistema en una frase

Universal Code Scanner es una **aplicación Flutter que lee códigos QR y de barras en el propio dispositivo**, interpreta su contenido en campos legibles, evalúa señales locales de riesgo y **exige confirmación explícita antes de ejecutar cualquier acción externa**. No consulta ningún servidor propio.

Para quién es esta documentación

Perfil	Empieza por	Después
Persona no técnica	Descripción general	Resumen ejecutivo
Desarrollador que se incorpora	Guía para nuevos desarrolladores	Instalación → Arquitectura → Mapa del código
Desarrollador con experiencia	Referencia técnica	Explicación profunda
Auditor técnico o de seguridad	Arquitectura	Seguridad → Riesgos y deuda técnica → Trazabilidad
Operación y soporte	Despliegue y operación	Solución de problemas
Otro agente de IA	Mapa del código	Referencia técnica + Trazabilidad

Tabla de contenidos

#	Documento	Contenido	Estado
01	Descripción general del sistema	Qué es, qué problema resuelve, actores, flujo, tecnologías y límites	Completo
02	Instalación y ejecución	Requisitos, dependencias, generación de plataformas, ejecución y pruebas	Completo
03	Arquitectura	Capas, patrones, diagramas, estado, errores, persistencia y seguridad	Completo
04	Mapa completo del código	Inventario jerárquico de directorios, archivos, clases y funciones	Completo
05	Referencia técnica	Catálogo de constantes, tipos, funciones, claves, almacenes y errores	Completo
06	Explicación profunda del código	Flujo interno, decisiones y casos límite de los módulos importantes	Completo

07	Base de datos	Motor, almacenes, diccionario de datos, migraciones y cifrado	Completo
08	Flujo de datos	Origen, validación, transformación, almacenamiento y salida de cada dato	Completo
09	APIs e integraciones	Contratos JSON, interfaces de plataforma y ausencia deliberada de red	Completo
10	Configuración	Preferencias, banderas, límites y consecuencias de cada valor	Completo
11	Seguridad	Cifrado, análisis de enlaces, validación de entrada y superficie de ataque	Completo
12	Pruebas y calidad	Suite existente, cobertura observable, huecos y propuesta priorizada	Completo
13	Despliegue y operación	CI/CD, artefactos, publicación, PWA, mantenimiento y reversión	Completo
14	Solución de problemas	Síntoma, causa, diagnóstico, solución y riesgo	Completo
15	Riesgos y deuda técnica	Hallazgos clasificados por severidad, impacto, evidencia y prioridad	Completo · informativo
16	Glosario	Términos técnicos, del dominio y del producto	Completo
17	Resumen ejecutivo	Presentación para decisión, sin detalle innecesario	Completo
18	Guía para nuevos desarrolladores	Itinerario, entorno, convenciones y primeras tareas	Completo
19	Matriz de trazabilidad	De la funcionalidad al código, a los datos y a su prueba	Completo

Los mismos documentos en PDF están en `pdf/`. Se generan desde estos Markdown con `python tool/build_system_documentation_pdf.py`; el procedimiento está en 13-deployment-and-operations.md.

Datos del análisis

Dato	Valor
Fecha del análisis	27 de agosto de 2026
Versión analizada	`1.1.0+2` (`pubspec.yaml`)

Commit analizado	`c1f9878`
Rama	`main`
Toolchain usado para verificar	Flutter 3.44.7 · Dart 3.12.2
Archivos versionados en el repositorio	171 (`git ls-files`)
Archivos Dart en `lib/`	59
Líneas de Dart en `lib/`	6 821
Archivos de prueba Dart	20 (19 en `test/`, 1 en `integration_test/`)
Casos de prueba declarados	58 · **57 los ejecuta `flutter test`**
Herramientas en `tool/`	12
Dependencias directas de producción	19 paquetes de pub + 2 del SDK
Señales locales de riesgo de URL	16
Tipos de contenido interpretados (`ContentKind`)	17
Almacenes Sembast	5

Convenciones

Estos marcadores aparecen a lo largo de los documentos y significan siempre lo mismo:

Marcador	Significado
Comprobado	Se leyó el código, se ejecutó el comando o se verificó el archivo en este repositorio
Inferencia	Conclusión razonada a partir del código, no una afirmación literal del repositorio
No identificado	Se buscó y no existe evidencia en ningún sentido
No documentado en el repositorio	Existe en el código pero ninguna documentación previa lo explic a
Requiere validación	Solo puede confirmarse con hardware, red o credenciales que e ste análisis no tiene

Otras convenciones:

- las rutas se escriben relativas a la raíz del repositorio;
- los nombres de clases, funciones, almacenes y claves se conservan **literales**, para que puedan buscarse con `grep`;
- ningún ejemplo contiene secretos, credenciales ni datos personales reales; se usan dominios reservados (`example.com`) y valores ficticios;
- los diagramas Mermaid siempre van acompañados de una explicación en texto: el diagrama nunca es el único portador de la información.

Qué se verificó ejecutando, y qué no

Ejecutado en la máquina de análisis con Flutter 3.44.7 sobre el commit `c1f9878`:

Comando	Resultado
`flutter pub get`	Resuelve · `pubspec.lock` versionado y sin cambios
`flutter analyze --fatal-infos`	**No issues found!** (100,8 s)
`flutter test`	Registrado en 12-testing-and-quality.md
`python tool/validate_structure.py`	Registrado en 12-testing-and-quality.md

Pendiente de validar

Lo que esta documentación **no** pudo comprobar en el entorno de análisis:

1. **Comportamiento en dispositivo físico.** Cámara real, biometría, almacén seguro del sistema, PDF grandes, TalkBack y VoiceOver solo pueden confirmarse en hardware. La matriz obligatoria vive en `../quality/DEVICE_TEST_MATRIX.md`.
2. **Compilaciones de iOS y macOS.** Requieren hardware Apple; la evidencia proviene del trabajo `apple` de la CI, no de esta máquina.
3. **Carpetas nativas.** `android/`, `ios/`, `web/` y `macos/` **no se versionan**: las genera `tool/bootstrap.py`. Lo que esta documentación describe de `ios/` y `macos/` proviene de leer ese generador, no de inspeccionar el resultado. `android/` y `web/` sí existen en la máquina de análisis, generados previamente, y se leyeron directamente.
4. **La lectura en un teléfono real después de las correcciones de 1.1.0.** Es la prueba que originó esas correcciones y sigue declarada pendiente en `../../VALIDATION.md`.
5. **La ejecución del catálogo de regresión de `test_assets/`.** Ninguna prueba Dart lo lee; ver 12-testing-and-quality.md y el hallazgo R-01 de 15-risks-and-technical-debt.md.

Relación con la documentación previa

Este directorio **no sustituye** la documentación existente del repositorio; la complementa y la enlaza. La documentación previa sigue siendo la fuente normativa en estas áreas:

Área	Fuente normativa
Estado de verificación de la versión publicada	`../../VALIDATION.md`
Qué está implementado y qué no	`../../IMPLEMENTATION_STATUS.md`
Historial de cambios por versión	`../../CHANGELOG.md`
Compromisos que no pueden romperse	`../quality/COMPATIBILITY_CONTRACT.md`
Comportamiento del escáner estado por estado	`../quality/SCANNER_UX.md`
Política de anclaje de dependencias	`../quality/LOCKFILE.md`
Modelo de amenazas y lista MASVS	`../security/THREAT_MODEL.md` · `../security/MASVS_CHECKLIST.md`

Lista de publicación en tiendas	`../RELEASE.md`